

Applied ML SP4 2025

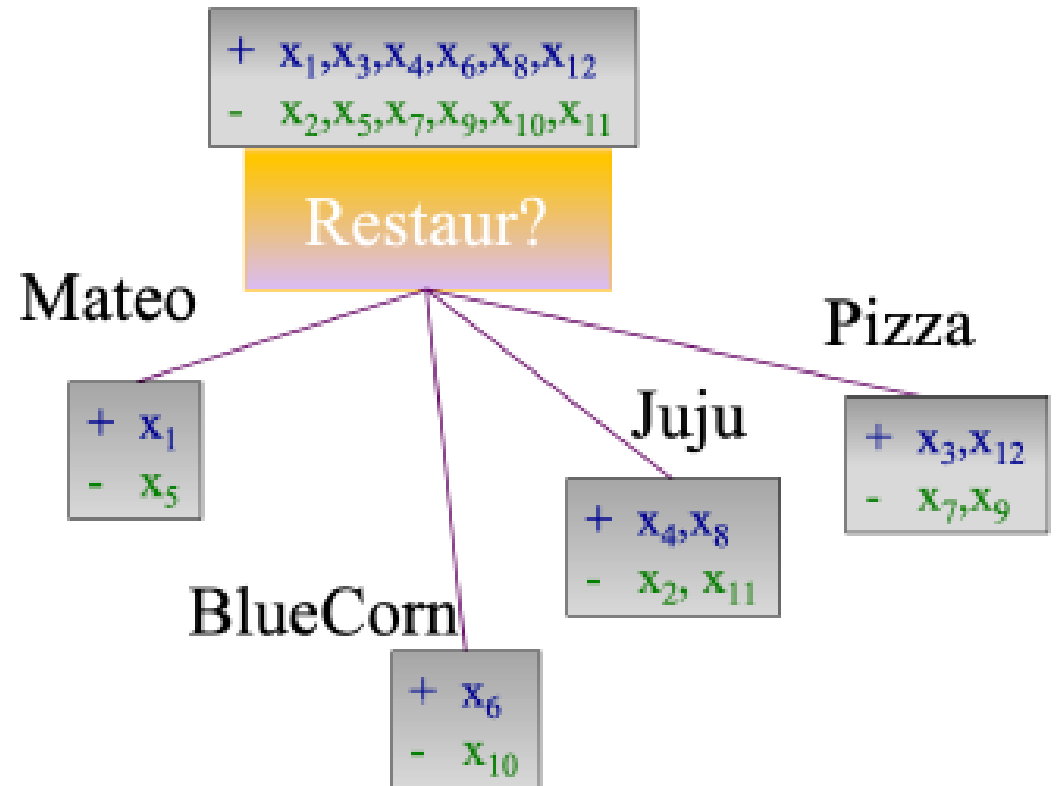
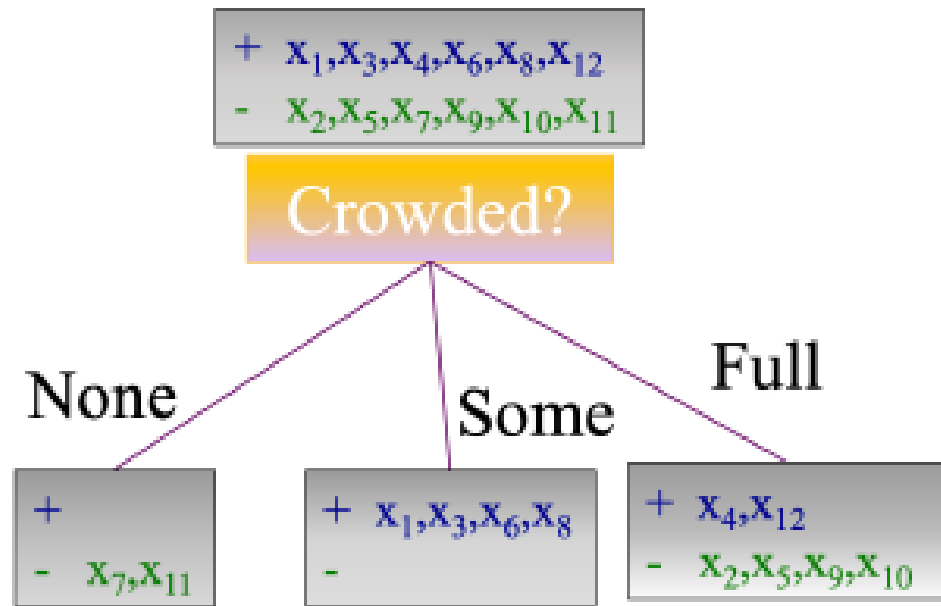
Ensemble and random forest

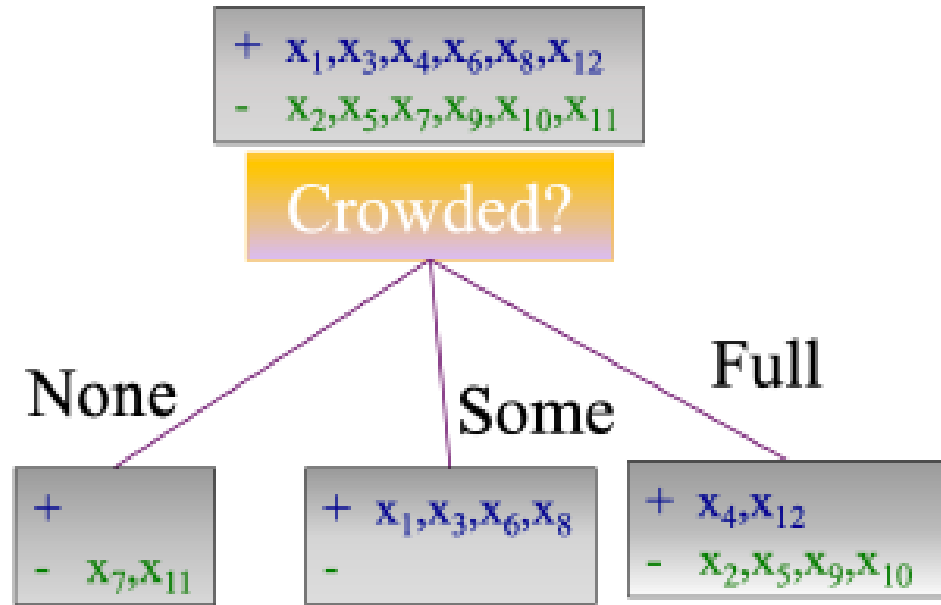
Selpi

Computer Science and Engineering

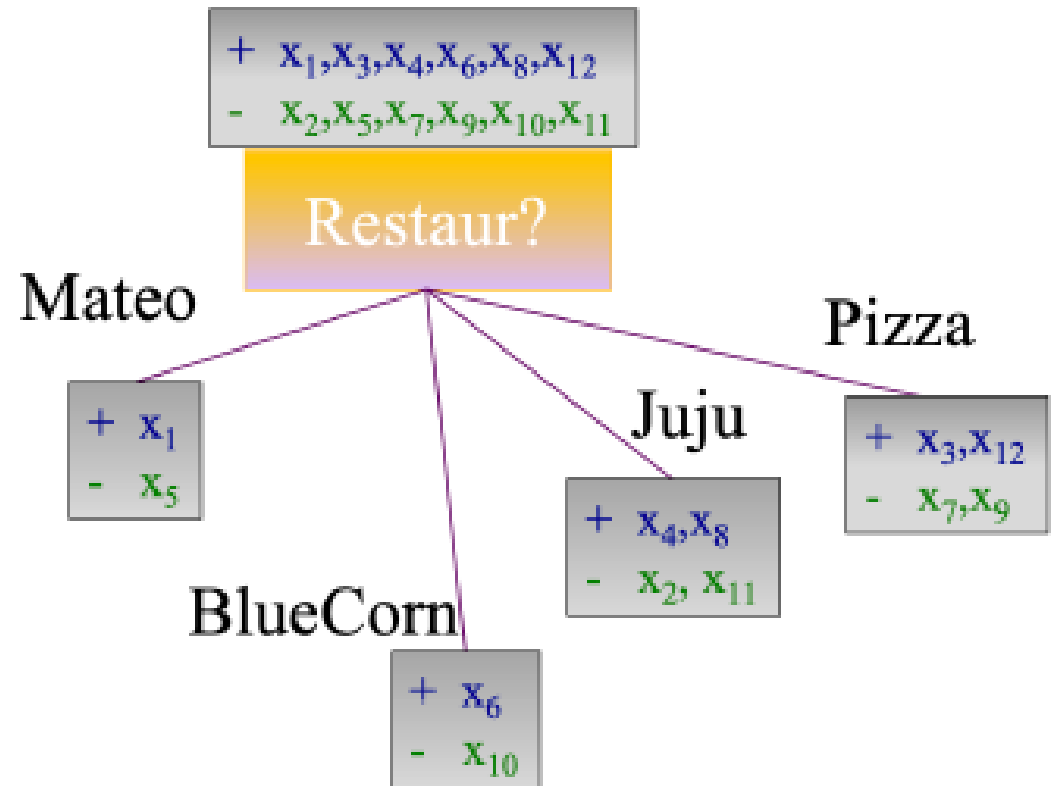
Chalmers and GU

Continuation of decision tree

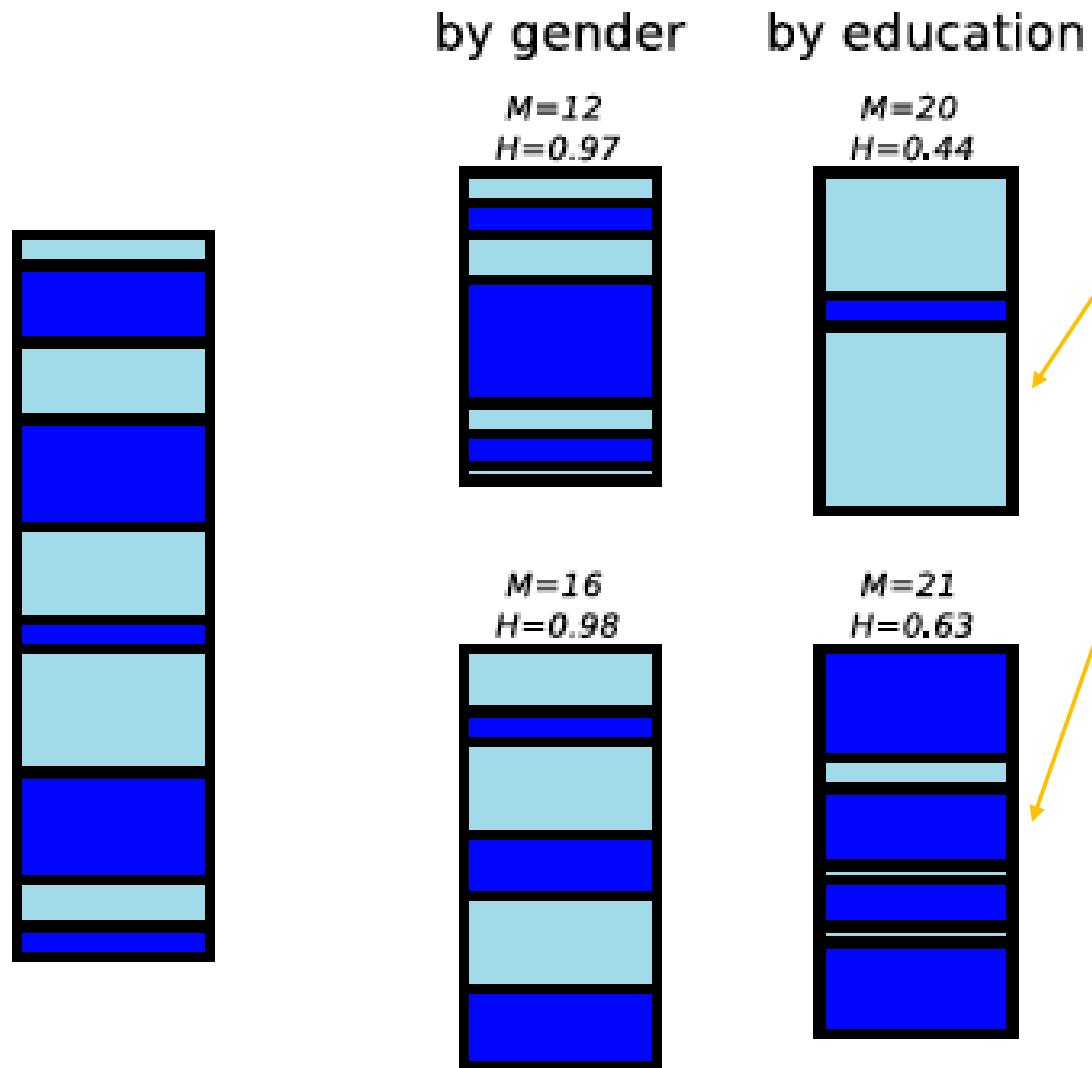




Useful feature split
makes the subsets
more homogeneous



Borrowed from example by Cynthia Rudin



Useful feature split
makes the subsets
more homogeneous

Borrowed from example by Richard Johansson

Learn a decision tree using recursive binary splitting

Data: Training data $\mathcal{T} = \{\mathbf{x}_i, y_i\}_{i=1}^n$

Result: Decision tree with regions $R_1, \dots, R_L$ and corresponding predictions $\hat{y}_1, \dots, \hat{y}_L$

- 1 Let R denote the whole input space
- 2 Compute the regions $(R_1, \dots, R_L) = \text{Split}(R, \mathcal{T})$
- 3 Compute the predictions $\hat{y}_\ell$ for $\ell = 1, \dots, L$ as

$$\hat{y}_\ell = \begin{cases} \text{Average}\{y_i : \mathbf{x}_i \in R_\ell\} & \text{(Regression problems)} \\ \text{MajorityVote}\{y_i : \mathbf{x}_i \in R_\ell\} & \text{(Classification problems)} \end{cases}$$

```
4 Function Split( $R, \mathcal{T}$ ):  
5   if stopping criterion fulfilled then  
6     return  $R$   
7   else  
8     Go through all possible splits  $x_j < s$  for all input variables  $j = 1, \dots, p$ .  
9     Pick the pair  $(j, s)$  that minimizes (2.5)/(2.6) for regression/classification problems.  
10    Split region  $R$  into  $R_1$  and  $R_2$  according to (2.4).  
11    Split data  $\mathcal{T}$  into  $\mathcal{T}_1$  and  $\mathcal{T}_2$  accordingly.  
12    return Split( $R_1, \mathcal{T}_1$ ), Split( $R_2, \mathcal{T}_2$ )  
13  end  
14 end
```

Predict from a decision tree

Data: Decision tree with regions $R_1, \dots, R_L$, training data $\mathcal{T} = \{\mathbf{x}_i, y_i\}_{i=1}^n$, test data point $\mathbf{x}_\star$

Result: Predicted test output $\hat{y}(\mathbf{x}_\star)$

- 1 Find the region R_ℓ which $\mathbf{x}_\star$ belongs to.
 - 2 Return the prediction $\hat{y}(\mathbf{x}_\star) = \hat{y}_\ell$.
-

- The members of that branch is all from the same class → return a leaf with that class
- Have reached max depth → return a leaf with majority class of the members of that branch
- See others in scikit learn

Shallow tree vs deep tree

For a given dataset, assume that a decision tree with depth 5 has a good performance.

- Imagine that the same dataset was described by a decision tree with depth 3.
- Compare the trees with depth 5 and depth 3.
- Now, imagine that we get a decision tree with depth 10 from the same dataset. We got several leaf nodes who have one member.
- Compare the trees with depth 10 and depth 5.

Shallow tree vs deep tree

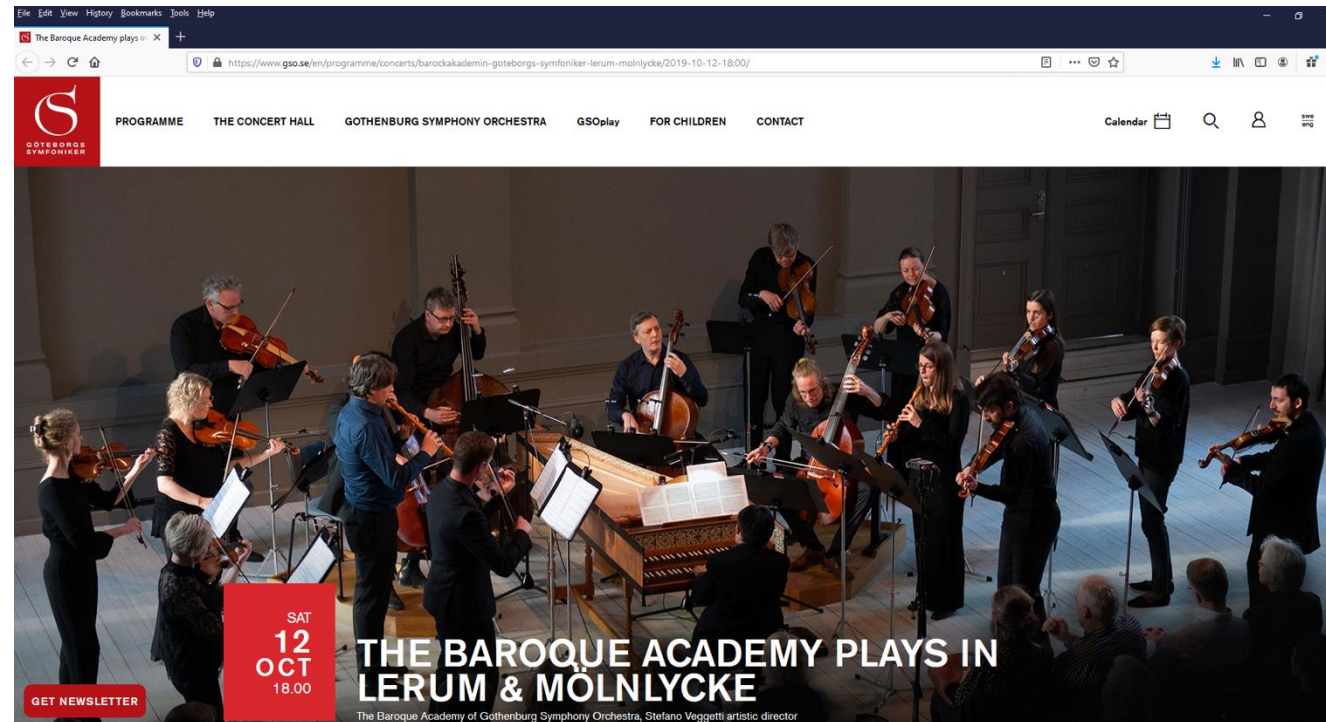
For a given dataset, assume that a decision tree with depth 5 has a good performance.

- Imagine that the same dataset was described by a decision tree with depth 3.
- Underfitting
- Bias
- Now, imagine that we get a decision tree with depth 10 from the same dataset. We got several leaf nodes who have one member.
- Overfitting to training data
- Might have low performance on test set
- Variance

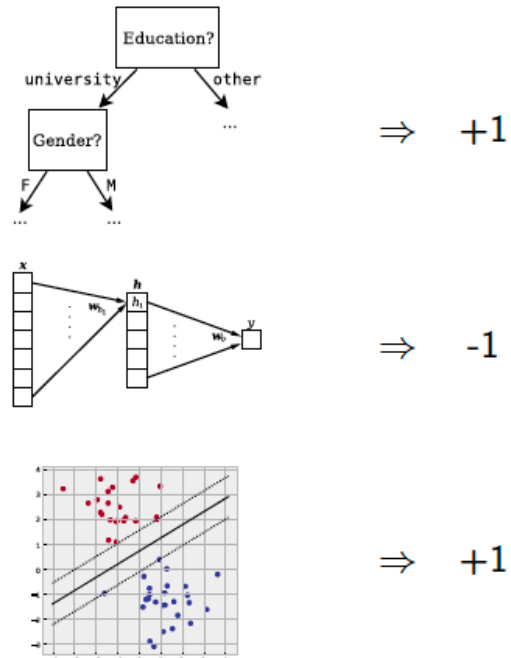
Ensemble

Ensembles

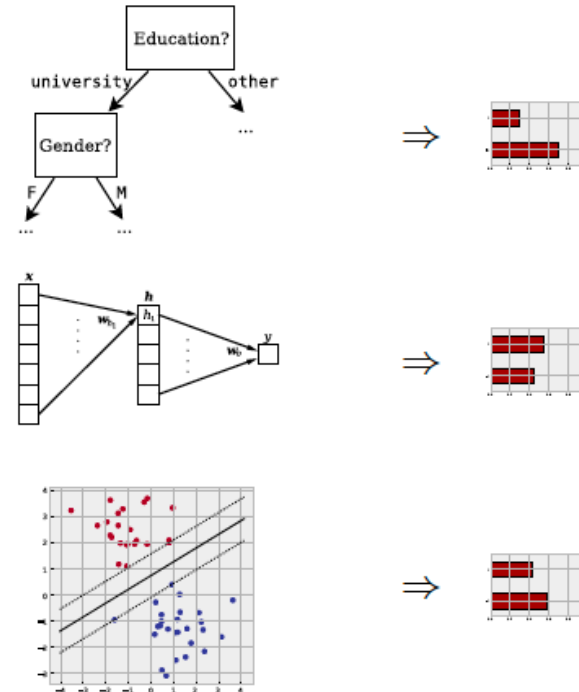
- Are ML models (classifiers, regressors, etc.) built by combining several models



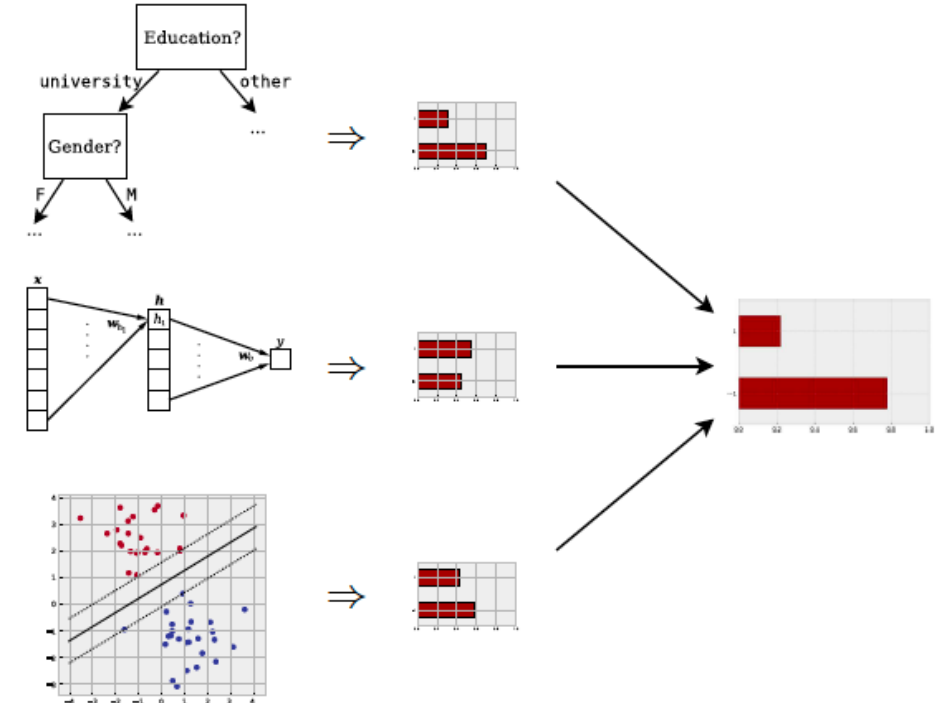
How ensemble is implemented?



Voting

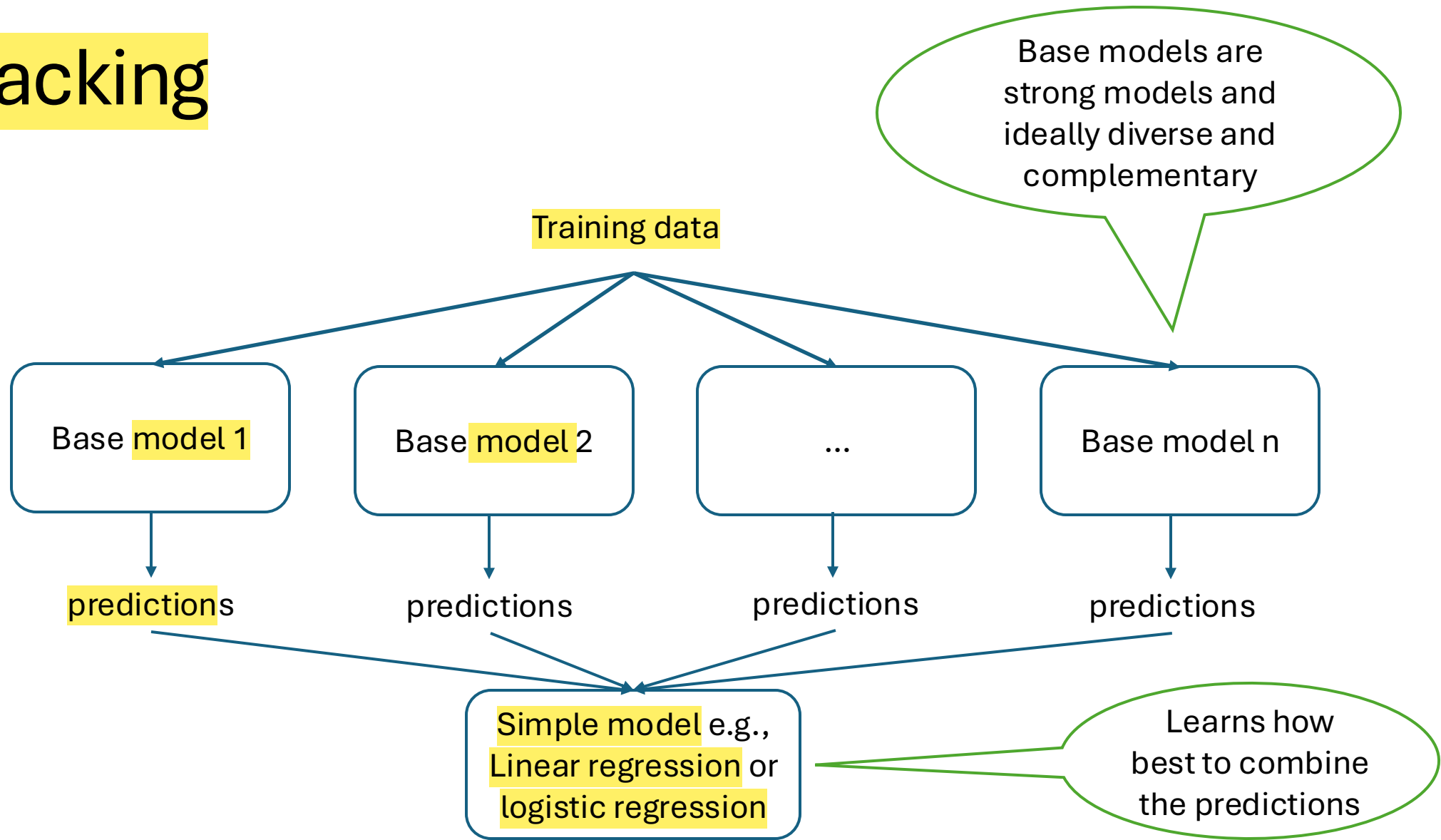


Averaging



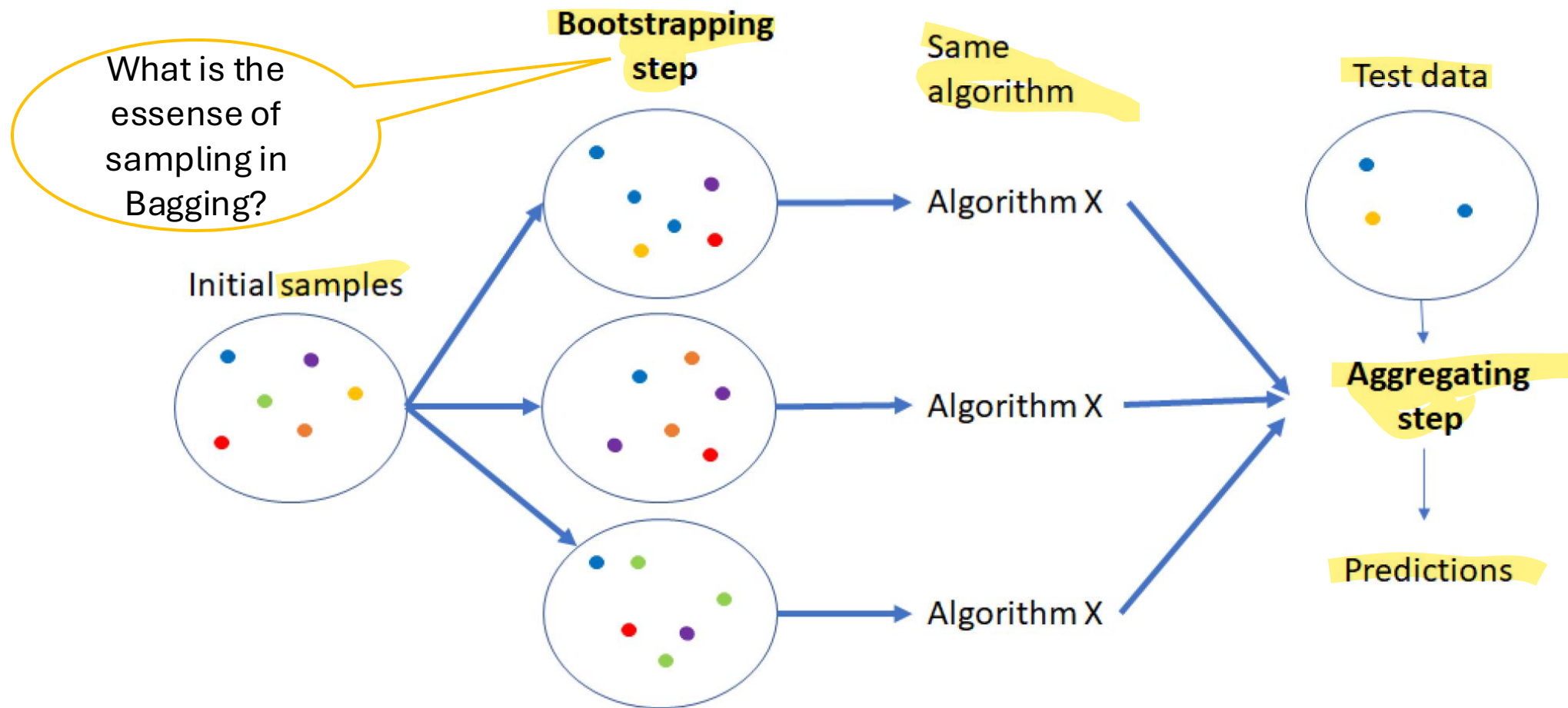
Stacking

Stacking



Random forest

Bagging (bootstrap aggregating, sampling with replacement)



Bagging

- How to create different training data, beside sampling with replacement on the samples?

Bagging

- How to create different training data, beside sampling with replacement on the samples?
- Pasting (sampling without replacement)
- Feature bagging (random subspace learning) – choosing random subsets of features

See Notebook

Generalisation

Error function in supervised ML

Define error function $E(\hat{y}, y)$ that compares $\hat{y}(\mathbf{x})$ to y as

Misclassification: $E(\hat{y}, y) \triangleq \mathbb{I}\{\hat{y} \neq y\} = \begin{cases} 0 & \text{if } \hat{y} = y \\ 1 & \text{if } \hat{y} \neq y \end{cases}$ (classification)

Squared error: $E(\hat{y}, y) \triangleq (\hat{y} - y)^2$ (regression)

ML task: Design a method that performs well given new, unseen data, i.e., achieve as small E_{new} as possible

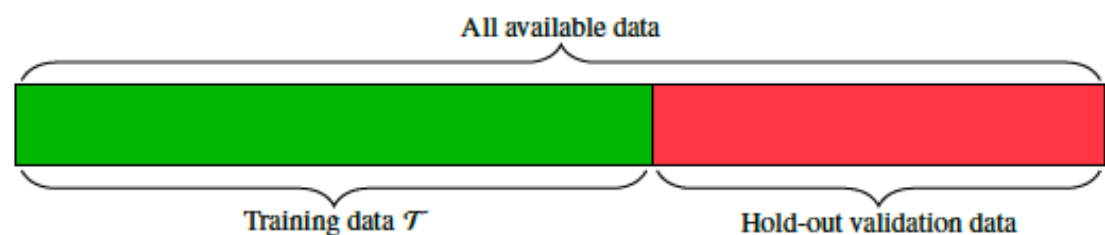
$$E_{new} \triangleq \mathbb{E}_{\star} [E(\hat{y}(\mathbf{x}_{\star}; \mathcal{T}), y_{\star})],$$

$$\mathbb{E}_{\star} [E(\hat{y}(\mathbf{x}_{\star}; \mathcal{T}), y_{\star})] = \int E(\hat{y}(\mathbf{x}_{\star}; \mathcal{T}), y_{\star}) p(\mathbf{x}_{\star}, y_{\star}) d\mathbf{x}_{\star} dy_{\star}.$$

$$E_{\text{train}} \triangleq \frac{1}{n} \sum_{i=1}^n E(\hat{y}(\mathbf{x}_i; \mathcal{T}), y_i), \text{ where } \{\mathbf{x}_i, y_i\}_{i=1}^n \text{ is the training data } \mathcal{T}.$$

Can we estimate
 E_{new} from
training data?

Estimating E_{new} from hold-out validation data



$$E_{\text{hold-out}} \triangleq \frac{1}{n_v} \sum_{j=1}^{n_v} E(\hat{y}(\mathbf{x}'_j; \mathcal{T}), y'_j).$$

With $\{\mathbf{x}'_j, y'_j\}_{j=1}^{n_v}$ is the hold-out validation data.

The variance of $E_{\text{hold-out}}$ decreases when the size of hold-out validation data increases. Big validation set means small training set, leads to bigger E_{new} .



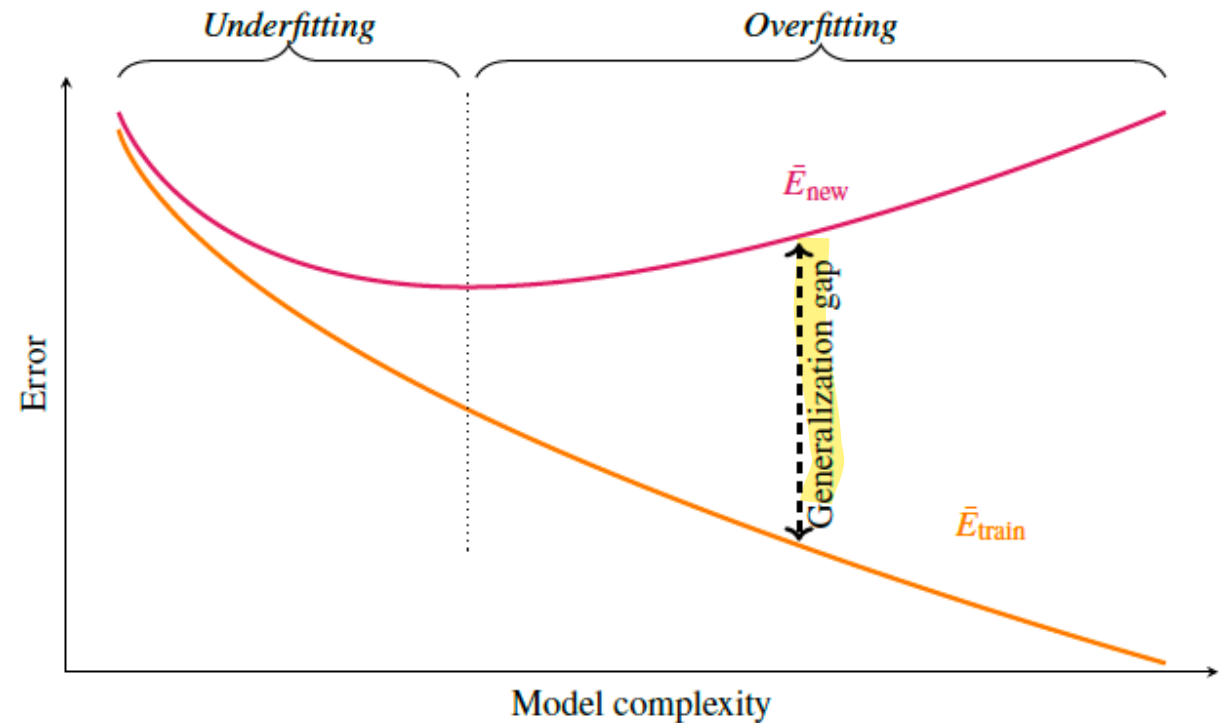
Cross-validation

Generalisation gap

If we train the model multiple times on different training sets:

$$\bar{E}_{\text{new}} \triangleq \mathbb{E}_{\mathcal{T}} [E_{\text{new}}(\mathcal{T})], \text{ and}$$

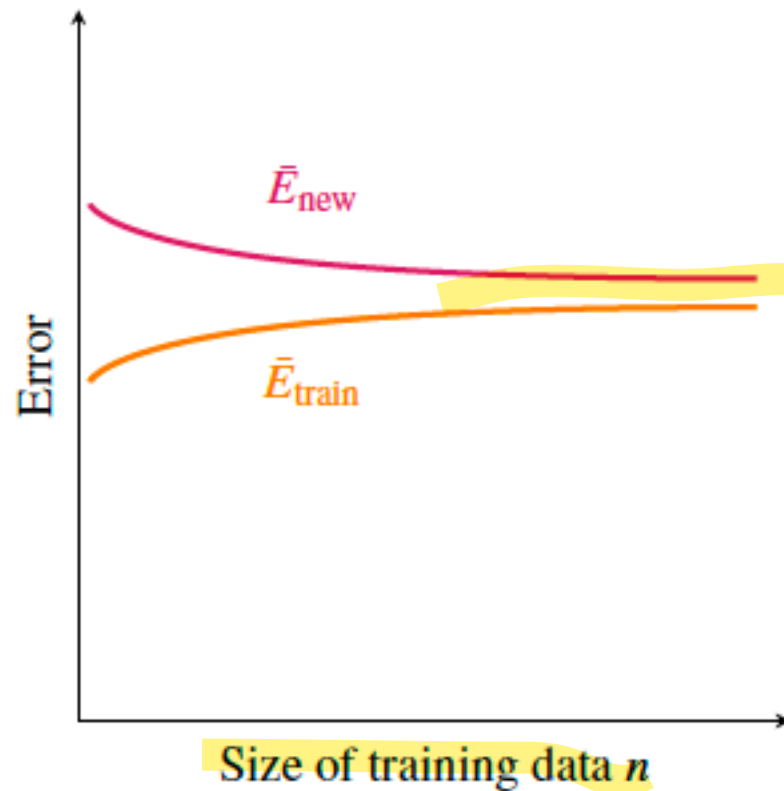
$$\bar{E}_{\text{train}} \triangleq \mathbb{E}_{\mathcal{T}} [E_{\text{train}}(\mathcal{T})].$$



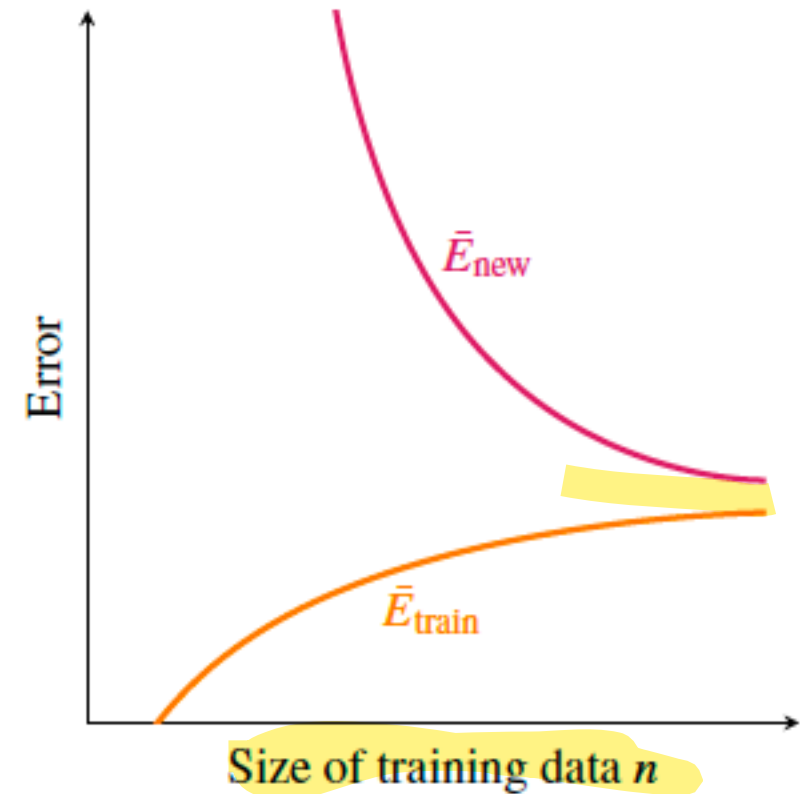
$$\bar{E}_{\text{train}} < \bar{E}_{\text{new}},$$

$$\text{generalization gap} \triangleq \bar{E}_{\text{new}} - \bar{E}_{\text{train}}.$$

Generalisation gap and size of training data



(a) Simple model



(b) Complex model

Next class session

- Data-centric vs Model-centric AI
- Data collection and annotation
- Imbalanced data
- Hyperparameter tuning